Practice 3: Light curve

Camila Cárdenas Uribe*
Universidad de Antofagasta, Antofagasta, Chile

(Dated: June 30, 2025)

This report presents the analysis and visualization of light curves from the TESS satellite using Python, as a solution to Practice 3. The main objectives include reproducing the graphs from Practice 1 using the real observational data provided in Practice 2, identifying possible outliers, and calculating the basic statistical properties of the light curves. This work seeks to reinforce the concepts seen in class on the implementation of libraries and functions, in addition to correcting bugs presented in Practice 2.

I. INTRODUCTION

As in Lab 1 and Lab 2, it is important to consult the light curve data obtained by the Transiting Exoplanet Survey Satellite (TESS). This is a space telescope developed by NASA and launched in April 2018. Its primary mission is to photometrically map the sky to detect exoplanets using the transit method [4]. The method identifies planets as they pass in front of their stars, causing small decreases in their brightness, giving rise to light curve graphs.

Light curves are one of the most fundamental tools in observational astronomy, as they provide information on the variation in the brightness of astronomical objects over time. In this lab, our goal is to analyze light curves with Python to deepen our understanding of the time-domain data.

Building on the previous exercises, the first objective is to reproduce the graphs from Exercise 1, now using the observational light curve files obtained in Exercise 2 as LC files. The second objective is to identify and highlight extreme outliers in the light curves, implementing Python-based methods (astropy library) for their detection. Finally, we compute basic statistical descriptors by implementing libraries such as numpy to quantitatively characterize the light curves. This report is thus structured to reflect the step-by-step approach taken to complete the exercise tasks, including visualizations, code snippets, and interpretation of the results.

* Institutional email: camila.cardenas.uribe@uamail.cl

II. METHODOLOGY AND RESULTS

This section describes the solutions for tasks 1 to 3 proposed for this practice with a detailed description of the codes used and an overview of the responses provided by the program.

A. Light Curve .csv to .lc correction

To automate the processing of CSV files from Practice 1, we wrote a Bash script that:

1. Changes the delimiter from commas to spaces.
2. Extracts only the relevant columns for plotting light curves: `TIME`, `PDCSAP_FLUX` and `PDCSAP_FLUX_ERR` .
3. Renames the file extension from `.csv` to `.lc`.

In practice 2, an error occurred when converting .csv files to .lc. This error consisted of the omission of the PDCSAP_FLUX_ERR column, essential for plotting the light curve, in addition to a data conversion error. After reviewing the documentation, it was detected that the last error in the code was due to the presence of NaN (Not a Number) values in the .csv files (blank spaces in the table), which could affect the behavior of awk [1]. Since in the first section we changed the delimiter to spaces, these non-numeric values were not considered, cutting off the column extensions. This produced an erroneous interpretation of the data, considering that the columns were incorrectly indexed, since awk did not correctly detect the column number, resulting in incorrect values when interpreting the

content as numbers. To correct this, the following was implemented:

```
#!/bin/bash
# Loop through all files endingin .csv
for archive in *.csv; do
    # Create a new file name
    # with a .lc extension
    new_archive="${archive%.csv}.lc"

    # Replace commas with spaces
    awk -F',' '
    # Extract the columns
    BEGIN {
        OFS = " "
    }
    NR==1 {
        for (i = 1; i <= NF; i++) {
            gsub(/^ +| +$/, "", $i)  #remove
                leading/trailing spaces
            if ($i = = "TIME") time= i
            if ($i = = "PDCSAP_FLUX") flux = i
            if ($i = = "PDCSAP_FLUX_ERR") err=i
        }
    }
    NR > 1 && time && flux && err {
        print $time, $flux, $err
    }
    ' "$archive" > "$new_archive"

    echo "File: $archive -> $new_archive"
done
```

First, it loops through all files in the current directory ending in .csv, which in this case are located in the CSV_File folder of the repository [1]. For each file, it creates a new name by changing the extension to .lc. Then, it uses awk to modify the file content: it sets the comma (',') as the field delimiter ('-F','), and defines the space ('" "') as the new output separator ('OFS'). By default, awk separates output fields with spaces, but defining it explicitly with the Begin function ensures that even if the system has a different configuration, the extracted values (in this case, the TIME, PDCSAP_FLUX and PDCSAP_FLUX_ERR columns) will be written separated by a single space instead of commas or another delimiter. The gsub instruction removes leading and trailing spaces for missing numeric values, preventing column indexing errors. In awk, the special variable NR stands for "Number of Records", that is, the current line number being processed while NF stands for "Number of Fields" and represents the number of columns or "fields" in the current line, determined by the delimiter that has been defined [2]. From there, print only the values corresponding to these tree columns in each row, removing the others, and save the result in the new .lc file. Finally, print a message to the terminal indicating that the file was processed successfully. All new .lc files are fixed in the LC_File folder within the repository.

[1] https://github.com/CamilaCU1613/Astroinformatics_Practice.git

```
3285.804512931147 2078.908447265625 11.244133949279785
3285.805901823592 2083.70263671875 11.255919456481934
3285.8072907165038 2065.45068359375 11.24039363861084
3285.8086796094153 2098.1396484375 11.260112762451172
3285.810068502326 2070.363037109375 11.242572784423828
3285.8114573952375 2082.028564453125 11.252561569213867
3285.812846288149 2079.14453125 11.240100860595703
3285.814235180594 2065.834716796875 11.23572826385498
3285.815624073505 2084.989013671875 11.237153053283691
3285.8170129664168 2074.001953125 11.242371559143066
3285.8184018593283 2076.243896484375 11.235358238220215
3285.819790752239 2055.9990234375 11.213791847229004
3285.8211796451506 2085.91943359375 11.243473052978516
```

FIG. 1. Output of .lc files restricted to the columns associated with the light curve plot: TIME, PDCSAP_FLUX and PDCSAP_FLUX_ERR.

B. Light curve plotting

Having the .lc files corrected from practice 2, the light curve graphs made with TOPCAT will now be repeated but now from Python:

```
# import libraries
import matplotlib.pyplot as plt
import glob

# Find all .fits files in the current directory
fits_files = glob.glob("*.lc")

for filename in fits_files:
    # Upload the file, assuming it is a .txt or
        .lc file with space separators
    df = pd.read_csv(filename, delim_whitespace
        =True)

    # remove NaN
    df = df.dropna(subset=['PDCSAP_FLUX'])

    # plot
    plt.figure(figsize=(10, 5))
    plt.errorbar(df['TIME'], df['PDCSAP_FLUX'],
         yerr=df['PDCSAP_FLUX_ERR'], markersize
        =3, fmt='.', ecolor='gray', alpha=0.4,
        color='dimgray')
    plt.xlabel('TIME/BJD - 2457000, days)')
    plt.ylabel('PDCSAP_FLUX/ e-/S')
    # remove the .lc extension from the object
        name
    plt.title(filename.replace("_lc.lc", ""))
    plt.grid(False)
    plt.legend()
    plt.tight_layout()
    plt.savefig(filename.replace("_lc.lc", ".
        png"))
    plt.show()
```

First, the necessary libraries are imported: matplotlib.pyplot for plotting and glob for searching for files. Using glob.glob("*.lc"), all available .lc files are retrieved. Then, for each file found, its contents are read using pandas.read_csv() with whitespace separators (delim_whitespace=True). Rows containing missing values (NaN) in the PDCSAP_FLUX column, which were processed in the correction above, are removed. Next, a light curve graph is generated using plt.errorbar(), showing the flux (PDCSAP_FLUX) as a function of time (TIME) with error bars (PDCSAP_FLUX_ERR). The graph is colored dark gray with partial transparency (alpha=0.4). The axes are labeled appropriately, and the title of each graph corresponds to the file name without the .lc extension. Finally, the graph is saved as a .png image within the repository in the LightCurve_python folder with the same base name as the original file and is also displayed on screen.

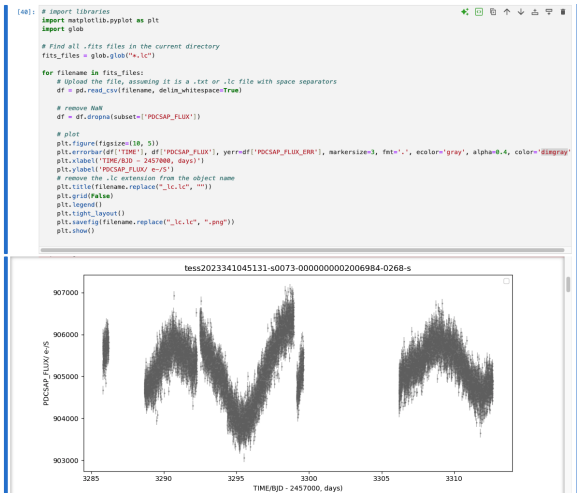


FIG. 2. Visualization in jupyter notebook.

C. Outliers

In this part of the practice, we seek to identify the ouleyers within the light curves obtained in the previous step. To do this, we will use the astropy library, expanding our knowledge seen in class with the sigma clipping statistical test seen within the statistics class program. Sigma clipping is an iterative statistical method used to identify and remove outliers from a dataset [3]. Given a dataset $\{x_i\}$, the procedure works as follows:

1. Compute the mean μ and standard deviation σ of the data.
2. Identify and mask the data points that satisfy:
$$|x_i - \mu| > k \cdot \sigma$$
where k is a chosen threshold (commonly $k = 3$).
3. Recompute the mean and standard deviation excluding the masked points.
4. Repeat the process for a fixed number of iterations or until convergence (no new points are excluded).

The result is a dataset in which statistical outliers are excluded, allowing for more robust analysis of the underlying trends.The code presented here combines the code worked on for sigma in the statistics class using the sigma_clip function and the code presented in the previous section for the light curve plots [3].

```
# import libraries
import matplotlib.pyplot as plt
import glob
import pandas as pd
from astropy.stats import sigma_clip
import numpy as np

# Find all .fits files in the current directory
fits_files = glob.glob("*.lc")

for filename in fits_files:
    # Upload the file, assuming it is a .txt or
        .lc file with space separators
    df = pd.read_csv(filename, delim_whitespace
        =True)

    # Remove NaN
    df = df.dropna(subset=['PDCSAP_FLUX'])

    # Apply sigma clipping to detect outliers
    flux_clipped = sigma_clip(df['PDCSAP_FLUX
        '], sigma=3, maxiters=5)
    outliers = flux_clipped.mask  # Boolean
        mask: True = outlier

    # Plot
    plt.figure(figsize=(10, 5))

    # Plot normal points
    plt.errorbar(df['TIME'][~outliers], df['
        PDCSAP_FLUX'][~outliers],
                 yerr=df['PDCSAP_FLUX_ERR'][~
                     outliers], fmt='.',
                 markersize=3, ecolor='gray',
                     alpha=0.4, color='dimgray
                     ', label='Normal')

    # Plot outliers in red
    plt.errorbar(df['TIME'][outliers], df['
        PDCSAP_FLUX'][outliers],
                 yerr=df['PDCSAP_FLUX_ERR'][
                     outliers], fmt='o',
```

```
                markersize=4, color='red',
                    alpha=0.8, label='Outliers
                    ')

plt.xlabel('TIME / (BJD - 2457000), days')
plt.ylabel('PDCSAP_FLUX / e-/s')
plt.title(filename.replace("_lc.lc", ""))
plt.grid(False)
plt.legend()
plt.tight_layout()
plt.savefig(filename.replace("_lc.lc", "
    _outliers.png"))
plt.show()
```

The code begins by importing the necessary libraries, including matplotlib for graphing, pandas for data manipulation, and sigma_clip from astropy.stats to detect outliers. The script finds all .lc files in the current directory and reads them as a space-delimited table, just like in the previous step. It removes rows with missing PDCSAP_FLUX values and applies sigma clipping with a threshold of 3 standard deviations for 5 iterations to isolate anomalous flux values. A Boolean mask identifies outliers, which are plotted in red on a time vs flux plot, while normal points are plotted in gray. The plot is appropriately labeled, saved as a PNG file with a name based on the original filename in the repository in the Outliers folder, and displayed.

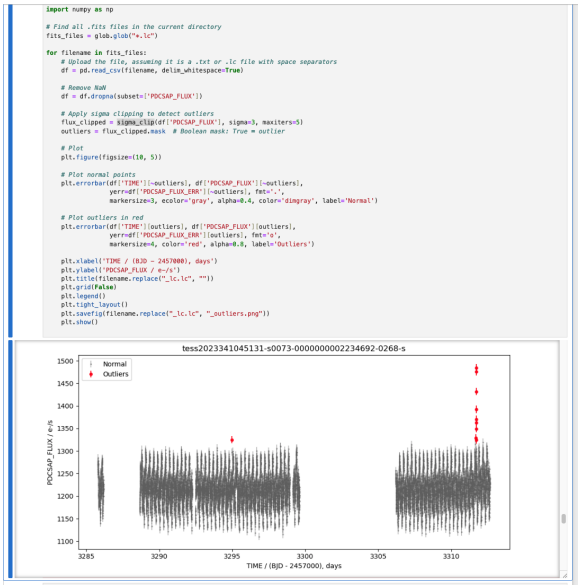


FIG. 3. Visualization in jupyter notebook.

D. Basic statistics

In this section, we analyze a sample of 20 light curves obtained from TESS observations, with the aim of identifying periodic variability potentially associated with exoplanet transits or stellar activity. For each target, we computed three basic statistical metrics: the flux amplitude, the standard deviation, and the most likely period derived using the Lomb-Scargle periodogram.

```
import matplotlib.pyplot as plt
import glob
import pandas as pd
import numpy as np
from astropy.timeseries import LombScargle

# Find all .lc files in the current directory
fits_files = glob.glob("*.lc")

for filename in fits_files:
    # Upload the file
    df = pd.read_csv(filename, sep=r"\s+")

    # Remove NaN
    df = df.dropna(subset=['PDCSAP_FLUX'])

    # Basic statistics
    amplitude = df['PDCSAP_FLUX'].max() - df['
        PDCSAP_FLUX'].min()
    std_dev = df['PDCSAP_FLUX'].std()
    print(f"\n{filename}")
    print(f"Amplitude: {amplitude:.2f}")
    print(f"Standard Deviation: {std_dev:.2f}")

    time = df['TIME']
    flux = df['PDCSAP_FLUX']

    # Lomb-Scargle periodogram
    frequency, power = LombScargle(time, flux).
        autopower()
    best_frequency = frequency[np.argmax(power)
        ]
    period = 1 / best_frequency
    print(f"Period: {period:.8f} days")

    # Phase folding
    phase = (time % period) / period
    phase_double = np.concatenate([phase, phase
         + 1])
    flux_double = np.concatenate([flux, flux])

    # Plot both periodogram and folded light
        curve
    fig, axs = plt.subplots(2, 1, figsize=(10,
        8), sharex=False)

    # Periodogram
    axs[0].plot(1 / frequency, power, color='
        blue', lw=1)
    axs[0].axvline(period, color='red',
        linestyle='--', label=f'Best period = {
        period:.5f} d')
    axs[0].set_xlabel('Period (days)')
```

```
axs[0].set_ylabel('Lomb-Scargle Power')
axs[0].set_title(f'Lomb-Scargle Periodogram
    : {filename.replace("_lc.lc", "")}')
axs[0].legend()
axs[0].grid(True)

# Folded light curve
axs[1].scatter(phase_double, flux_double, s
    =5, alpha=0.6, color='black')
axs[1].set_xlabel('Phase')
axs[1].set_ylabel('PDCSAP_FLUX / e-/s')
axs[1].set_title(f'Phase-folded Light Curve
    : {filename.replace("_lc.lc", "")}')
axs[1].grid(True)

# Save figure
output_filename = filename.replace("_lc.lc
    ", "_analysis.png")
plt.tight_layout()
plt.savefig(output_filename, dpi=300)
plt.close()
```

These parameters provide a first-order characterization of the variability and help discriminate between genuine periodic signals and artifacts due to instrumental noise or data sampling I.

File	Amplitude (e^-/s)	Std Dev (e^-/s)	Period (days)
1	143.42	16.27	0.00138866
2	4048.50	635.96	0.00138865
3	56.94	7.24	4.40158092
4	918.08	196.14	0.59533570
5	72334.00	12934.37	0.08806041
6	2073.72	452.05	2.46327067
7	207.01	34.30	0.58242133
8	4279.00	426.96	0.00138853
9	274512.22	47145.40	0.00138980
10	271.84	37.05	0.00139167
11	953.51	214.28	3.31477045
12	36842.50	7036.64	1.60776335
13	185.17	25.87	0.00138865
14	313.28	48.72	0.43375779
15	49.96	6.78	0.00138889
16	374.61	36.42	0.272032930
17	205.45	22.69	0.001389222
18	137.56	16.31	0.001388891
19	659.91	65.10	0.001388475
20	548.45	58.45	0.001384766

TABLE I: Statistical metrics for 20 light curves.

- **Flux Amplitude** (A): The flux amplitude is defined as the range between the maximum and minimum flux values of the light curve:

$$A = F_{\max} - F_{\min}$$

This metric provides a rough measure of the total flux variation over peak-to-peak time. Note that we do not divide by two to find the intrinsic amplitude of the curve, as we want to characterize the separation between peaks [2]. Transit photometry here can capture the depth of the transit if the light curve is dominated by a single periodic event. However, if the signal is dominated by noise or intrinsic stellar variability of the host star, the amplitude may reflect other physical or instrumental effects. In our dataset, the amplitude ranges from approximately 50 e^-s to over 270,000 e^-s. Higher amplitudes, for example, in File 9, are typically associated with noisy or crowded signals and may not indicate true transits.

- **Standard Deviation of Flux** (σ) The standard deviation is computed as:

$$\sigma = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(F_i - \bar{F})^2}$$

where F_i is the flux at time t_i, and $\bar{F}$ is the mean flux. This value quantifies the dispersion of the data and can indicate how variable the traffic is, regardless of any periodicity. The standard deviation of the flux values across the 20 TESS lightcurves varies significantly, from as little as ~ 6.78 e^-/s to over $\sim$ 47,000e^-/s. This metric reflects the overall dispersion or variability in each lightcurve and is a strong indicator of intrinsic stellar activity or observational noise. Low standard deviations (below $\sim 30e^-/s$ generally correspond to photometrically quiet stars or well-behaved lightcurves, where transit signals, if present, might be easier to detect. Conversely, high standard deviations (above $1000e^-/s$) often indicate instrumental systematics, contamination, or unresolved variability unrelated to exoplanet transits. These large dispersions reduce sensitivity to surface traffic signals and can lead to false detections or artificially boosted periodogram peaks. Interestingly, many of the curves with very short (nonphysical) periods also exhibit high standard deviations, suggesting that noise dominates the signal and corrupts the period estimate.

- **Estimated Period** (P)

The period is estimated using the Lomb–Scargle periodogram, which is well suited for unevenly sampled data like those from TESS. It searches for the frequency f that maximizes the power of the signal in frequency space:

$$P = \frac{1}{f_{\max}}$$

The estimated periods in our sample fall into two categories:

- Very short periods around ∼0.00139 days (∼2 minutes)
- Physically plausible periods from ∼0.27 to ∼4.4 days

Among the 20 light curves analyzed, the estimated periods show a clear bimodal distribution. The vast majority (15 out of 20) have periods tightly clustered around 0.00139 days, corresponding to less than 2 minutes. These short timescales are well below the physical limits of exoplanet orbital motion and are inconsistent with any known type of transiting system. These values are most likely spurious, caused by noise, instrumental artifacts, or aliasing due to the TESS cadence. In contrast, a smaller subset of five objects (IDs 3, 4, 6, 11, 12, and 14) have periods ranging from approximately 0.4 to 4.4 days. These are within the range expected for hot Jupiters and short-period binary systems and are therefore considered physically plausible. These five candidates warrant further modeling and monitoring, while the remaining periods should be treated with caution or excluded from transit-based analyses. We could justify some of the results observed in the table I given the nature of the data, where TESS typically operates with two cadences [4]:

- 2-minute cadence (for preselected targets)
- 30-minute Full Frame Images (FFI)

With a 2-minute cadence, the Nyquist period is approximately 4 minutes (twice the sampling rate). Therefore, signals with periods close to or below this threshold (e.g., ∼ 2 minutes) are susceptible to:

- Spectral leakage
- Aliasing effects
- Overfitting by the periodogram

Therefore, these suspiciously short periods are artifacts and should not be interpreted as real transits or astrophysical periodicities. The high standard deviation of the data, as we saw in the previous section, prevents the periods initially taken by Lombscargle from being the most reliable. However, if a more detailed visual inspection is performed in cases where the peak-to-peak amplitude indicates a transit, it is possible to obtain a reasonable period and even manage to phase the light curve with:

$$\phi = \frac{t \bmod P}{P}$$

Where t is the time of each observation and P is the estimated period. The modulo operation wraps the timestamps in the interval $[0, P)$, effectively aligning all cycles into a single period. Plotting the flow as a function of ϕ reveals any consistent periodic behavior, such as transits or pulsations, regardless of the absolute timing, as shown in Figure x and in the repository examples saved in the Statistical_Analysis folder.

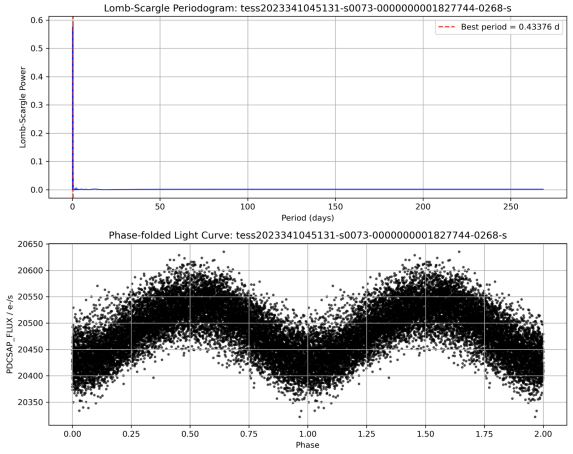


FIG. 4. Visualization in Jupyter Notebook for the fourteenth object corresponding to ID: tess2023341045131-s0073-0000000001827744-0268-s.

III. CONCLUSION

This practice successfully implemented a comprehensive workflow to process and analyze TESS light curve data using Python. From raw CSV files, we implemented scripts to correct data formatting issues and isolate essential columns. Light curves were

plotted with error bars, outliers were identified using sigma clipping, and key statistical measures such as flux amplitude, standard deviation, and period were calculated for each object. Period analysis revealed clear bimodal behavior: while some light curves displayed physically plausible periods (from 0.4 to 4.4 days), most exhibited unrealistically short periods of around 0.00139 days, likely due to noise, aliasing, or limitations in the TESS timing. This highlights the importance of cross-validating periodogram results using visual inspection and physical constraints. Phase-folded curves and Lomb-Scargle periodograms helped confirm whether the periodic variability was real or spurious.

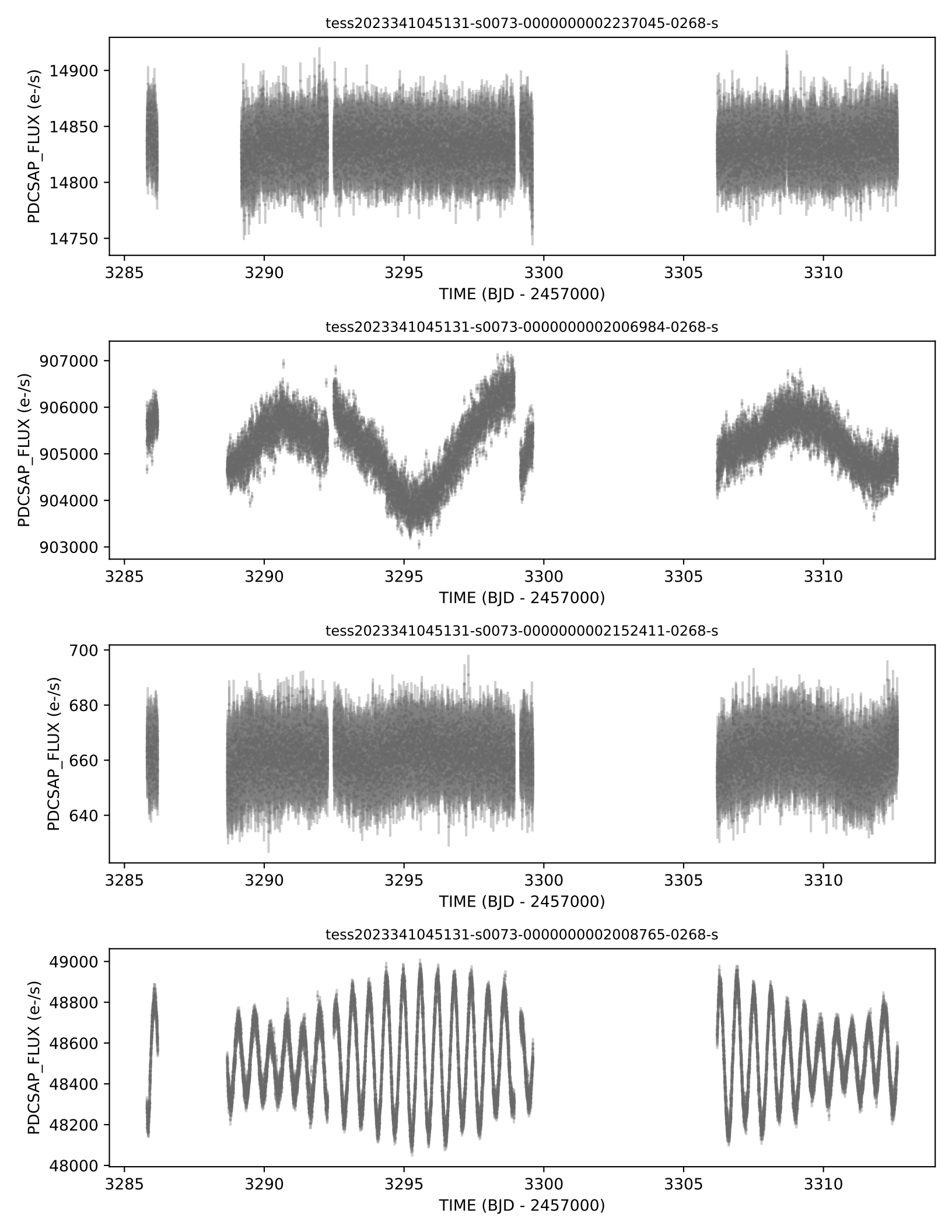
---

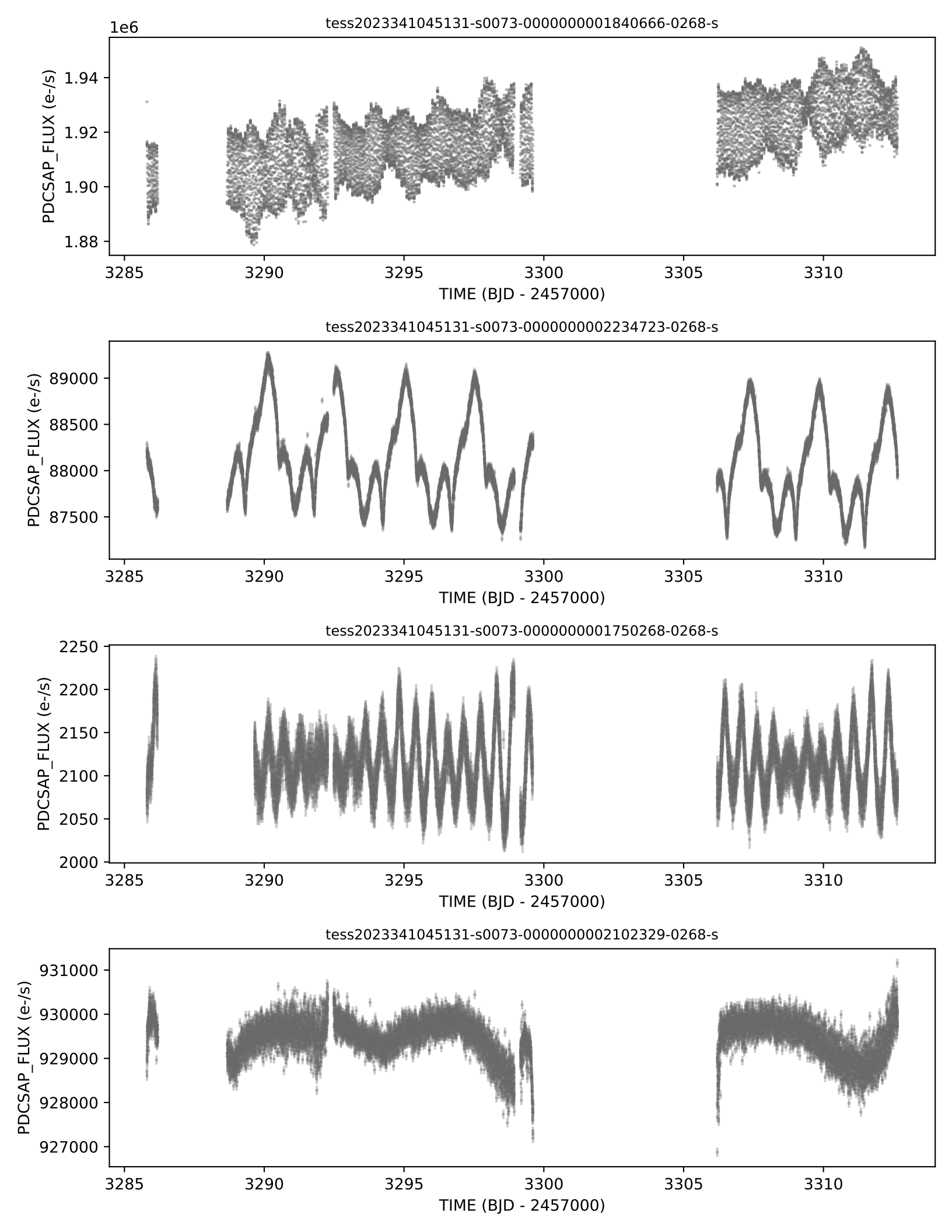
[1] Diane, B. C., Robbins, A. D., Paul, H. R., Richard, S., and Piet, v. O. (1993). The awk manual. *Free Software Foundation*.

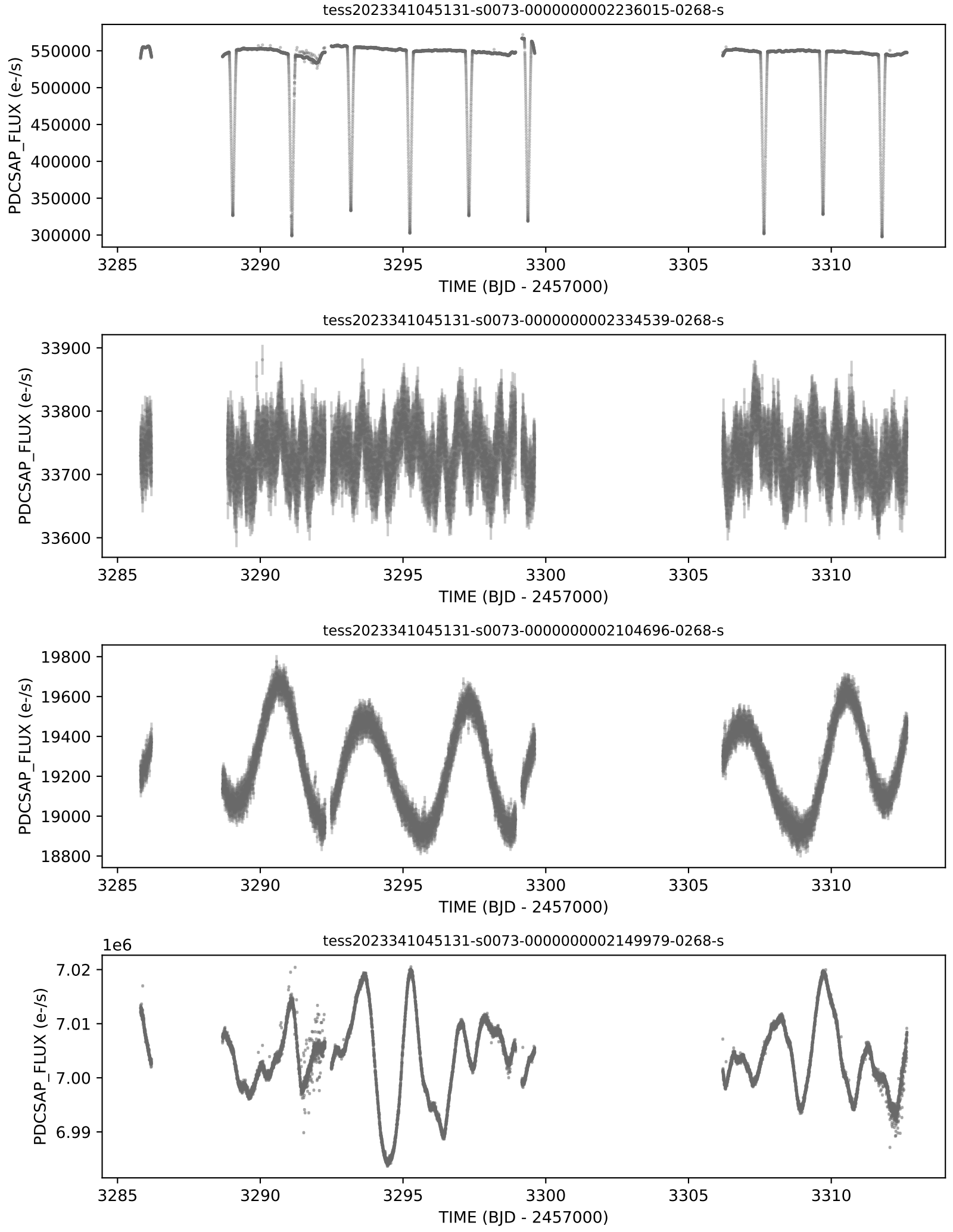
[2] GeeksforGeeks (2025). AWK command in Unix/Linux with examples.

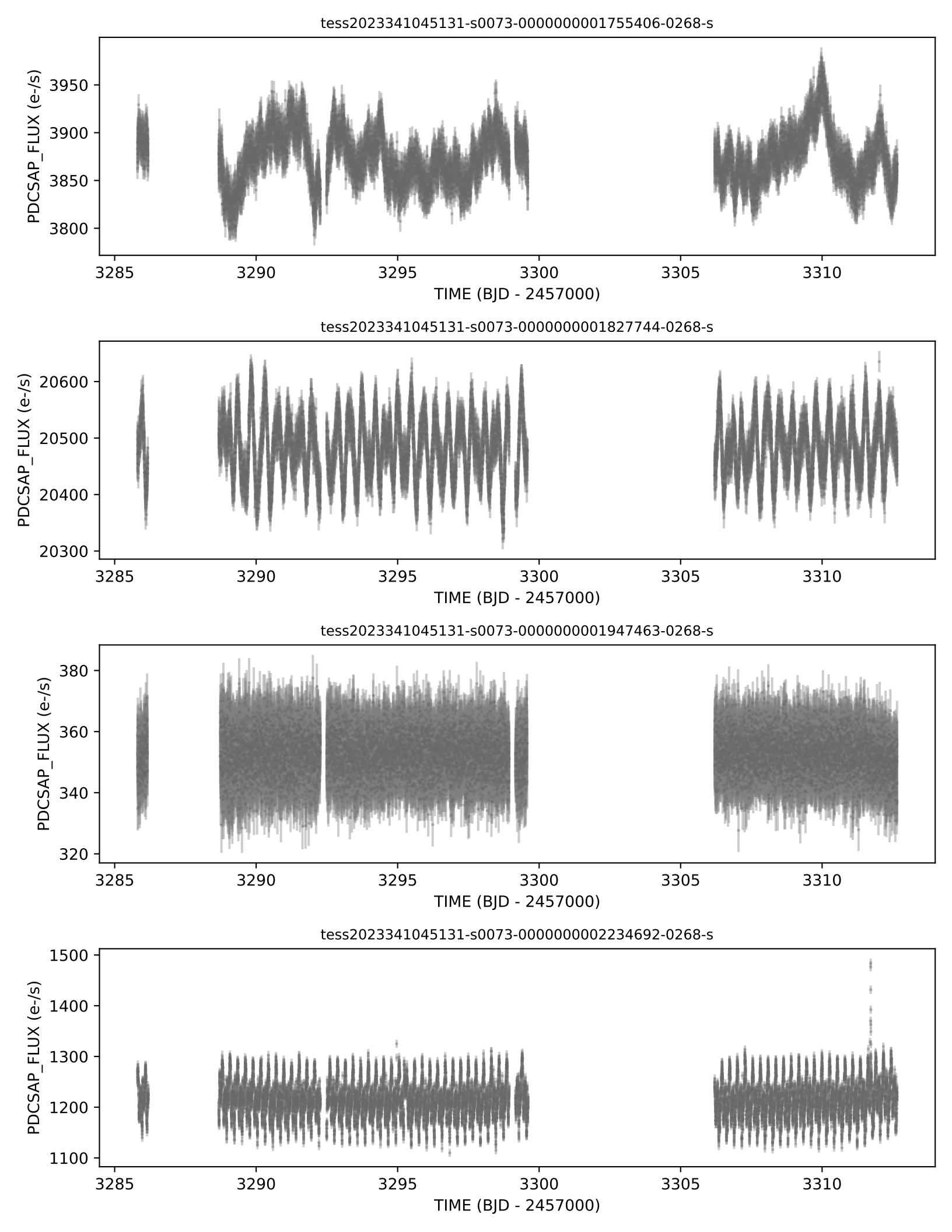
[3] Lomb, N. R. (1976). Least-squares frequency analysis of unequally spaced data. *Astrophysics and Space Science*, 39(2):447–462.

[4] Ricker, G. R., Latham, D. W., Vanderspek, R. K., Ennico, K. A., Bakos, G., Brown, T. M., Burgasser, A. J., Charbonneau, D., Clampin, M., Deming, L. D., Doty, J. P., Dunham, E. W., Elliot, J. L., Holman, M. J., Ida, S., Jenkins, J. M., Jernigan, J. G., Kawai, N., Laughlin, G. P., Lissauer, J. J., Martel, F., Sasselov, D. D., Schingler, R. H., Seager, S., Torres, G., Udry, S., Villasenor, J. N., Winn, J. N., and Worden, S. P. (2018). Transiting Exoplanet Survey Satellite (TESS). 215:450.06.


tess2023341045131-s0073-0000000002237045-0268-s
PDCSAP_FLUX (e-/s)
14900
14850
14800
14750
3285
3290
3295
3300
3305
3310
TIME (BJD - 2457000)
tess2023341045131-s0073-0000000002006984-0268-s
PDCSAP_FLUX (e-/s)
907000
906000
905000
904000
903000
3285
3290
3295
3300
3305
3310
TIME (BJD - 2457000)
tess2023341045131-s0073-0000000002152411-0268-s
PDCSAP_FLUX (e-/s)
700
680
660
640
3285
3290
3295
3300
3305
3310
TIME (BJD - 2457000)
tess2023341045131-s0073-0000000002008765-0268-s
PDCSAP_FLUX (e-/s)
49000
48800
48600
48400
48200
48000
3285
3290
3295
3300
3305
3310
TIME (BJD - 2457000)


tess2023341045131-s0073-0000000001840666-0268-s
1e6
PDCSAP_FLUX (e-/s)
1.94
1.92
1.90
1.88
3285
3290
3295
3300
3305
3310
TIME (BJD - 2457000)
tess2023341045131-s0073-0000000002234723-0268-s
PDCSAP_FLUX (e-/s)
89000
88500
88000
87500
3285
3290
3295
3300
3305
3310
TIME (BJD - 2457000)
tess2023341045131-s0073-0000000001750268-0268-s
PDCSAP_FLUX (e-/s)
2250
2200
2150
2100
2050
2000
3285
3290
3295
3300
3305
3310
TIME (BJD - 2457000)
tess2023341045131-s0073-0000000002102329-0268-s
PDCSAP_FLUX (e-/s)
931000
930000
929000
928000
927000
3285
3290
3295
3300
3305
3310
TIME (BJD - 2457000)


tess2023341045131-s0073-0000000002236015-0268-s
PDCSAP_FLUX (e-/s)
550000
500000
450000
400000
350000
300000
3285
3290
3295
3300
3305
3310
TIME (BJD - 2457000)
tess2023341045131-s0073-0000000002334539-0268-s
PDCSAP_FLUX (e-/s)
33900
33800
33700
33600
3285
3290
3295
3300
3305
3310
TIME (BJD - 2457000)
tess2023341045131-s0073-0000000002104696-0268-s
PDCSAP_FLUX (e-/s)
19800
19600
19400
19200
19000
18800
3285
3290
3295
3300
3305
3310
TIME (BJD - 2457000)
tess2023341045131-s0073-0000000002149979-0268-s
1e6
PDCSAP_FLUX (e-/s)
7.02
7.01
7.00
6.99
3285
3290
3295
3300
3305
3310
TIME (BJD - 2457000)


tess2023341045131-s0073-0000000001755406-0268-s
PDCSAP_FLUX (e-/s)
3950
3900
3850
3800
3285
3290
3295
3300
3305
3310
TIME (BJD - 2457000)
tess2023341045131-s0073-0000000001827744-0268-s
PDCSAP_FLUX (e-/s)
20600
20500
20400
20300
3285
3290
3295
3300
3305
3310
TIME (BJD - 2457000)
tess2023341045131-s0073-0000000001947463-0268-s
PDCSAP_FLUX (e-/s)
380
360
340
320
3285
3290
3295
3300
3305
3310
TIME (BJD - 2457000)
tess2023341045131-s0073-0000000002234692-0268-s
PDCSAP_FLUX (e-/s)
1500
1400
1300
1200
1100
3285
3290
3295
3300
3305
3310
TIME (BJD - 2457000)